

TYPES OF INHERITANCE

1. Single Inheritance

Structure: Animal → Dog

Steps

1. Create a package named singleinheritance.
2. Create parent class Animal.
3. Create child class Dog using extends.
4. Create Main class.
5. Create a Dog object.
6. Call both methods.

Program

```
package singleinheritance;
```

```
// Parent class
```

```
class Animal {
```

```
    // Parent class method
```

```
    void eat() {
```

```
        System.out.println("Animal eats");
```

```
    }
```

```
}
```

```
// Child class inherits Animal
```

```
class Dog extends Animal {
```

```
    // Child class method
```

```
    void bark() {
```

```
        System.out.println("Dog barks");
```

```
    }
```

```

}

// Main class
public class Main {

    public static void main(String[] args) {

        // Creating object of Dog
        Dog d = new Dog();

        // Calling inherited method
        d.eat();

        // Calling Dog method
        d.bark();
    }
}

```

Output :

```

Animal eats
Dog barks

```

2. Multilevel Inheritance

Structure: Animal → Dog → Puppy

Steps

1. Create package multilevelinheritance.
2. Create parent class Animal.
3. Create Dog that extends Animal.
4. Create Puppy that extends Dog.
5. Create Main class.
6. Create a Puppy object.
7. Call all three methods.

Program

```
package multilevelinheritance;
```

```
// First parent class
```

```
class Animal {
```

```
    // Method of Animal
```

```
    void eat() {
```

```
        System.out.println("Animal eats");
```

```
    }
```

```
}
```

```
// Dog inherits Animal
```

```
class Dog extends Animal {
```

```
    // Method of Dog
```

```
    void bark() {
```

```
        System.out.println("Dog barks");
```

```
    }
```

```
}
```

```
// Puppy inherits Dog
```

```
class Puppy extends Dog {
```

```
    // Method of Puppy
```

```
    void play() {
```

```
        System.out.println("Puppy plays");
```

```
    }
```

```
}
```

```
// Main class
```

```
public class Main {
```

```
public static void main(String[] args) {  
  
    // Creating object of Puppy  
    Puppy p = new Puppy();  
  
    // Calling Animal method  
    p.eat();  
  
    // Calling Dog method  
    p.bark();  
  
    // Calling Puppy method  
    p.play();  
}  
}
```

Output

```
Animal eats  
Dog barks  
Puppy plays
```

3. Hierarchical Inheritance

Structure:

```
Animal  
 /  \  
Dog  Cat
```

Steps

1. Create package hierarchicalinheritance.
2. Create parent class Animal.
3. Create Dog extending Animal.

4. Create Cat extending Animal.
5. Create Main class.
6. Create objects of Dog and Cat.
7. Call their methods.

Program

```
package hierarchicalinheritance;
```

```
// Parent class
```

```
class Animal {
```

```
    // Common method
```

```
    void eat() {
```

```
        System.out.println("Animal eats");
```

```
    }
```

```
}
```

```
// First child class
```

```
class Dog extends Animal {
```

```
    // Dog method
```

```
    void bark() {
```

```
        System.out.println("Dog barks");
```

```
    }
```

```
}
```

```
// Second child class
```

```
class Cat extends Animal {
```

```
    // Cat method
```

```
    void meow() {
```

```
        System.out.println("Cat meows");
```

```
    }  
}  
  
// Main class  
public class Main {  
  
    public static void main(String[] args) {  
  
        // Creating Dog object  
        Dog d = new Dog();  
  
        // Calling inherited method  
        d.eat();  
  
        // Calling Dog method  
        d.bark();  
  
        // Creating Cat object  
        Cat c = new Cat();  
  
        // Calling inherited method  
        c.eat();  
  
        // Calling Cat method  
        c.meow();  
    }  
}
```

Output

```
Animal eats  
Dog barks  
Animal eats  
Cat meows
```

4. Multiple Inheritance Using Interfaces

Java **does not support multiple inheritance using classes**. It can be achieved using **interfaces**.

Structure:

```
A   B  
 \  /  
  C
```

Steps

1. Create package multipleinheritance.
2. Create interface A.
3. Create interface B.
4. Create class C.
5. Use implements A, B.
6. Implement the methods.
7. Create Main class.
8. Create object of C.

Program

```
package multipleinheritance;
```

```
// First interface
```

```
interface A{
```

```
    // Method declaration
```

```
    void show();
```

```
}
```

```
// Second interface
```

```
interface B {
```

```
    // Method declaration
```

```
    void display();
```

```
}
```

```
// Class implements both interfaces
```

```
class C implements A, B {
```

```
    // Implementing A's method
```

```
    public void show() {
```

```
        System.out.println("This is A method");
```

```
    }
```

```
    // Implementing B's method
```

```
    public void display() {
```

```
        System.out.println("This is B method");
```

```
    }
```

```
}
```

```
// Main class
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Creating object of C
```

```
        C obj = new C();
```

```
        // Calling A method
```

```
        obj.show();
```

```
// Calling B method  
obj.display();  
}  
}
```

Output

```
This is A method  
This is B method
```

5. Hybrid Inheritance

Hybrid inheritance is a **combination of two or more types of inheritance**.

Java does not support hybrid inheritance directly using classes, but it can be achieved using **interfaces**.

Steps

1. Create package hybridinheritance.
2. Create interface A.
3. Create interface B extending A.
4. Create interface C.
5. Create class D implementing B and C.
6. Create Main class.
7. Create object of D.
8. Call all methods.

Program

```
package inh;
```

```
//Single Inheritance(Animal->Dog)
```

```
class Animal1 {
```

```
void eat() {
```

```
    System.out.println("Animal eats food");
```

```
}
```

```
}
```

```
class Dog1 extends Animal1 {
```

```
void bark() {
```

```
    System.out.println("Dog barks");
```

```
}
```

```
}
```

```
//Multilevel Inheritance(Animal->Dog->Puppy)
```

```
class Animal2 {
```

```
void eat() {
```

```
    System.out.println("Animal eats food");
```

```
}
```

```
}
```

```
class Dog2 extends Animal2 {
```

```
void bark() {
```

```
    System.out.println("Dog barks");
```

```
}
```

```
}
```

```
class Puppy2 extends Dog2 {
```

```
void play() {
```

```
    System.out.println("Puppy plays");
```

```
}  
}
```

```
//Hierarchial Inheritance( Animal )  
//          /  \  
//      Dog   Cat
```

```
class Animal3 {
```

```
void eat() {  
    System.out.println("Animal eats food");  
}  
}
```

```
class Dog3 extends Animal3 {
```

```
void bark() {  
    System.out.println("Dog barks");  
}  
}
```

```
class Cat3 extends Animal3 {
```

```
void meow() {  
    System.out.println("Cat meows");  
}  
}
```

```
//Hybrid Inheritance(Multilevel+Hierarchial)  
//Hierarchial:Animal->Dog,Animal->Cat
```

//Multilevel:Animal->Dog->Puppy

```
class Animal4 {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}
```

//First child of Animal

```
class Dog4 extends Animal4 {
```

```
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

//Second child of Animal

```
class Cat4 extends Animal4 {
```

```
    void meow() {  
        System.out.println("Cat meows");  
    }  
}
```

//Puppy inherits Dog

//Therefore: Animal -> Dog -> Puppy

```
class Puppy4 extends Dog4 {
```

```
    void play() {  
        System.out.println("Puppy plays");  
    }  
}
```

```
}  
}
```

```
//Main Class
```

```
public class Hybrid {
```

```
public static void main(String[] args) {
```

```
    System.out.println("Single Inheritance");
```

```
    Dog1 d1 = new Dog1();
```

```
    d1.eat();
```

```
    d1.bark();
```

```
    System.out.println("Multilevel Inheritance");
```

```
    Puppy2 p2 = new Puppy2();
```

```
    p2.eat();
```

```
    p2.bark();
```

```
    p2.play();
```

```
System.out.println("Hierarchial Inheritance");
```

```
Dog3 d3 = new Dog3();
```

```
d3.eat();
```

```
d3.bark();
```

```
Cat3 c3 = new Cat3();
```

```
c3.eat();
```

```
c3.meow();
```

```
System.out.println("Hybrid Inheritance");
```

```
// Dog object
```

```
Dog4 d4 = new Dog4();
```

```
d4.eat();
```

```
d4.bark();
```

```
// Cat object
```

```
Cat4 c4 = new Cat4();
```

```
c4.eat();
```

```
c4.meow();
```

```
// Puppy object  
Puppy4 p4 = new Puppy4();  
  
p4.eat();  
p4.bark();  
p4.play();  
}  
}
```

Output

```
Animal eats food  
Dog barks  
Animal eats food  
Cat meows  
Animal eats food  
Dog barks  
Puppy plays
```